Term Project #1

202113935 이준혁

1) NAT은 어디에 구현되어 있는가?

NAT는 IP헤더, TCP/UDP 포트 번호 등을 직접 수정해야하고, 응답 패킷에 대한 복원, 즉 상태 기반 처리도 필요하기에 커널 모드에서 동작해야한다. 특히 NAT는 패킷을 중간에서 가로채어 각 구간 별로 필요한 작업을 진행하기에 패킷이 커널 네트워크 스택을 흐르는 중간에 가로챌 수 있는 Netfilter 프레임워크에 구현되어 있고 경로는 net/netfilter/nf_nat_*.c 이다.

2) Netfilter 프레임워크는 어떻게 패킷을 처리하는가?

먼저 Netfilter는 커널에서 진행중인 패킷을 가로채어 작업할 수 있는 프레임워크라고 하였다. 그럼 이 패킷을 언제 가로채어서 어떤 작업을 커널이 수행할 수 있는지 정의 해주어야 하는데, 이 '언제'에 해당하는 것이 Hook이다. Hook은 총 5로 구성되어 있고, 커널은 이 Hook 시점에서 패킷을 가로채어 각 Hook에 연결된 작업을 수행한다.

첫 번째 Hook은 NF_INET_PRE_ROUTING이다. 이는 패킷이 커널을 통과하기 전에 작동되어 커널이 라우팅 테이블을 보기 전에 수행된다. 주로 목적지 주소를 바꾸는 작업인 DNAT을 수행한다. NF_INET_PRE_ROUTING은 들어오는 모든 패킷에 대해 적용되고, 이 작업을 통과한 후에는 라우팅 결정을 받게 된다.

두 번째 Hook은 NF_INET_LOCAL_IN이다. 이 과정에서는 목적지가 내부, 즉 시스템 자체가 목적지인 패킷에 대해 수행하는 Hook이다. 주로 시스템에 직접 들어오는 패킷의 필터링을 담당한다. 필터링이라 함은, 해당 패킷의 IP주소, 포트 번호, 프로토콜 등을 토대로 패킷을 허용할지 거부할지 등을 정하는 것이다.

세 번째 Hook은 NF_INET_ROUTING이다. 이번엔 패킷의 목적지가 로컬 시스템이 아닌 본 시스템을 통과하여 다른 네트워크로 전달되어야 하는 경우에 수행된다. 즉 중간 라우팅 포인트로 작용할 때 수행되는 것이다. 여기서도 마찬가지로 패킷 필터링이 수행되는데, IP주소, 포트 번호 등을 토대로 패킷을 다른 네트워크로 보낼지, 말 지를 판단하는 과정이다.

네 번째 Hook은 NF_INET_LOCAL_OUT이다. 로컬 시스템에서 생성된 패킷이 네트워크 계층을 통과하기 전에 수행되는데, 여기서는 로컬에서 생성된 패킷의 필터링이 이루어 진다. 여기서도 동일하게 IP주소, 포트 번호 등을 토대로 해당 패킷이 목적지로의 전달을 허용, 거부등을 결정한다.

마지막으로 NF_INET_POST_ROUTING이다. 패킷이 커널을 떠나 NIC로 가기 직전에 수행된다. 이미 최종 라우팅이 결정된 뒤로 여기서는 출발지 주소를 해당 인터페이스의 주소로 변환을 하는 SNAT이 수행된다. 나가는 모든 패킷에 대하여 수행된다.

패킷을 가로채는 시점은 정해졌다. 그럼 이 패킷을 가로채면 어떤 작업을 해야할지도 정해줘야 한다. 아래 사진은 include/linux/netfilter.h의 nf_hook_ops 구조체이다. 해당 구조체에는 Hook에서 실행할 함수들을 기록할 수 있게 되어있다. 중요한 변수들을 보면 hook은 등록하려는 훅 콜백(함수), hooknum은 다섯가지 Hook(prerouting, postrouting ...)중 하나, priority는 해당 함수의 우선순위이다.

```
struct nf_hook_ops {
    struct list_head    list;
    struct rcu_head     rcu;

    /* User fills in from here down. */
    nf_hookfn       *hook;
    struct net_device   *dev;
    void            *priv;
    u8          pf;
    enum nf_hook_ops_type   hook_ops_type:8;
    unsigned int        hooknum;
    /* Hooks are ordered in ascending priority. */
    int         priority;
};
```

이들을 구조체로 정의한 후 include/linux/netfilter.h의 nf_register_net_hook(s)를 사용하여 등록을 한다. 그럼 각 Hook에서 실행될 함수들이 chain형태로 등록이 완료되어서 사용할 수 있게 되었다. 추후에 커널이 패킷을 가지고 작업을 하며 Hook포인트에 걸때 NF_HOOK()을 통해 nf_hook()을 호출하고 nf_hook() 내부의 nf_hook_slow()를 통해 실질적으로 등록된 nf_hook_ops 함수들을 실행한다.

이제 각 Hook에서 실행될 함수들을 연결해 어떤 작업을 할지도 알아 냈다. 그럼이번엔 nf_hook_slow()를 통해 실행되는 함수들은 어떠한 기준에 의해 허용, 차단, 목적지/출발지 수정등의 작업을 할 것이다. 여기서 기준이 되는 것은 iptable 규칙이다.

먼저 iptable은 Netfilter에 규칙을 저장하는 역할을 한다. 유저 모드에서 sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080
와 같은 명령어를 치면 커널에 전달되어 테이블: nat, 체인: PREROUTING, 조건: -p tcp --dport 80, 타켓: -j REDIRECT --to-port 8080으로 저장된다. 해당 규칙이 PREROUTING체인으로 지정되어 있으므로 PREROUTING Hook에 패킷이 들어오면 nf_hook_slow()를 통해 실행되는 함수들은 해당 규칙을 토대로 각자 작업을 수행한다.

3) 예제의 NAT처리 흐름

예제의 흐름은 로컬 프로세스가 만든 IPv4 패킷이 외부로 나가는 과정이다.
먼저 패킷이 IP계층으로 가는 과정을 보자. /net/ipv4/ip_output.c에서 ip_queue_xmit()에 의해 ip_local_out() -> __ip_local_out()이 호출된다. __ip_local_out()에서는 ip헤더 길이, 체크섬등을 설정하고 nf_hook()을 호출한다.

```
    return nf_hook(NFPROTO_IPV4, NF_INET_LOCAL_OUT,
                   net, sk, skb, NULL, skb_dst_dev(skb),
                   dst_output);
}
```

주요 인자들을 보면 NFPROTO_IPV4 : 프로토콜 , NF_INET_LOCAL_OUT : 훅 지점, dst_output : 규칙 검사 후 최종적으로 호출할 함수(ip_output()으로 연결됨) 가 있다.

Include/linux/netfilter.h 의 nf_hook()을 보면 nf_hook_slow()를 호출하는 흐름이 있다. 여기서 nf_hook_slow()가 체인된 함수들과 iptables 규칙을 이용해 실질적인 규칙 기반 검사를 수행한다.

```
if (hook_head) {
    struct nf_hook_state state;

    nf_hook_state_init(&state, hook, pf, indev, outdev,
                       sk, net, okfn);

    ret = nf_hook_slow(skb, &state, hook_head, 0);
}
```

Hook_head에는 해당 훅에서 실행할 함수들이 저장된 nf_hook_entries가 저장되어있어 실행할 함수 목록이 있다면 nf_hoow_slow()를 호출하여 작업을 수행한다.

Net/netfilter/core.c 의 nf_hoow_slow()의 일부이다.

```
for (; s < e->num_hook_entries; s++) {
    verdict = nf_hook_entry_hookfn(&e->hooks[s], skb, state);
    switch (verdict & NF_VERDICT_MASK) {
```

Num_hook_entries는 해당 훅에 연결된 함수들의 수이고 이 구문에서 연결된 함수들을 차례로 실행하여 반환되는 결과, ACCEPT, DROP, QUEUE, STOLEN 등의 값을 토대로 제어한다. ACCEPT가 나오면 계속해서 다음 훅 함수로 연결된다. 그럼 패킷을 내보내기 위한 NAT 테이블 훅이 여기에 걸려있을 텐데 한번 알아보자.

먼저 nat를 수행하는 함수로 nf_nat_inet_fn()이 실행된다. 해당 함수는 net/netfilter/nf_nat_core.c에 위치해 있다.

```
if (!nf_nat_initialized(ct, maniptype)) {
    struct nf_nat_lookup_hook_priv *lpriv = priv;
    struct nf_hook_entries *e = rcu_dereference(lpriv->entries);
    unsigned int ret;
    int i;

    if (!e)
        goto null_bind;

    for (i = 0; i < e->num_hook_entries; i++) {
        ret = e->hooks[i].hook(e->hooks[i].priv, skb,
                               state);
        if (ret != NF_ACCEPT)
            return ret;
        if (nf_nat_initialized(ct, maniptype))
            goto do_nat;
    }
```

해당 함수의 일부인데, Ct는 conntrack 객체이다. nat가 적용된 연결인지 새 연결인지 nf_nat_initialized를 통해 판단한 후 새 연결이라면 for문을 통해 훅 함수를 순회한다. 여기서 훅 함수들은 nat를 위한 nat테이블 훅이다. 여기서는 DNAT을 위한 작업을 수행한다. 먼저 xt_do_table() 호출하여 유저가 입력한 명령어를 토대로 규칙을 구조화 한다. 그 다음 함수로 xt_dnat_target_v2()를 통해 nf_nat_setup_info()를 실행한다. 해당 함수는 xt_do_table()에서 구조화된 규칙을 토대로 conntrack객체에 "이 패킷은 ~~의 주소로 목적지를 변경해야해!" 하고 상태를 기록한다. 즉, 실제 주소 변경이 아닌 변경되어야 한다고 상태를 기록하는 작업이다. 해당 작업을 마치고 리턴되어 if(nf_nat_initialized)에 걸리면 방금 nf_nat_setup_info()의 결과로 ct가 nat를 수행해야 한다고 저장되어 있으므로 do_nat레이블로 이동한다.

```
do_nat:
    return nf_nat_packet(ct, ctinfo, state->hook, skb);
```

해당 레이블에는 nf_nat_packet을 콜백하는데, 해당 함수는 동일한 파일에 있다.(net/netfilter/nf_nat_core.c)

```
enum nf_nat_manip_type mtype = HOOK2MANIP(hooknum);
```

```
if (mtype == NF_NAT_MANIP_SRC)
    statusbit = IPS_SRC_NAT;
else
    statusbit = IPS_DST_NAT;
```

Nf_nat_packet() 내부의 코드이다. hooknum에 따라 mtype이 설정된다. 여기서는 local_out이 되고, 따라서 statusbit = IPS_DST_NAT로 설정된다.

```
if (ct->status & statusbit)
    verdict = nf_nat_manip_pkt(skb, ct, mtype, dir);
```

그 후 해당 구문에서 nf_nat_manip_pkt를 통해 실제 destination 주소가 변경된다.

Net/netfilter/nf_nat_proto.c의 nf_nat_manip_pkt()을 알아보자.

```
if (!l4proto_manip_pkt(skb, iphdroff, hdroff, target, maniptype))
    return false;
iph = (void *)skb->data + iphdroff;

if (maniptype == NF_NAT_MANIP_SRC) {
    csum_replace4(&iph->check, iph->saddr, target->src.u3.ip);
    iph->saddr = target->src.u3.ip;
} else {
    csum_replace4(&iph->check, iph->daddr, target->dst.u3.ip);
    iph->daddr = target->dst.u3.ip;
}
return true;
```

(!l4proto_manip_pkt())를 통해 l4, tcp/udp 헤더의 포트번호를 수정하여 체크섬을 수정하고, 하단 (maniptype == NF_NAT_MANIP_DST)를 통해 ip헤더의 ip주소를 수정, 체크섬도 수정하고 최종적으로 tcp/udp의 헤더도 수정하여 변환을 마친다.

쭉쭉 리턴되어 nf_hook()으로 돌아가 dst_output, 즉 ip_output()이 호출되어 해당 함수에서 post_routing 훅으로 넘어가게 된다.